

SIC Lab

Datagram Transport Layer Security (DTLS)

version 1.1

Authors: André Zúquete, Vitor Cunha

Version log:

- 1.0: Initial version
- 1.1: Added examples using the WolfSSL Python module

1 Introduction

Transport Layer Security (TLS) runs over TCP because TCP provides a reliable, ordered, and error-checked stream of data between two endpoints, which is essential for TLS's security guarantees — encryption, integrity, and authentication. However, TCP is unsuitable in scenarios where low latency and fast session startup are critical, such as real-time communications (VoIP, video/audio streaming) and WebRTC. TCP and its retransmission mechanism also makes it not ideal for VPN tunnels and the handshake also affects many IoT applications (e.g., one-way links). Because of this, Datagram Transport Layer Security (DTLS) was created to provide TLS-like security for datagram-based communications (i.e., UDP).

2 Create Certificate Chain (X.509)

TLS and DTLS require that we have a set of cryptographic key pairs to authenticate the participants (i.e., servers and clients). Unlike in the previous guide where OpenVPN performs mutual authentication, that is both client and server authenticate each other, in TLS (and DTLS by extension) only the server needs to authenticate itself to the client. When the client also authenticates itself to the server, we call that protocol mTLS (Mutual TLS), which is usually supported by most modules that implement TLS in your favorite programming language.

You may use the certificates and keys generated in the previous guide (i.e., the ones for OpenVPN). Nevertheless, in order to create new certificates you can do so using OpenSSL or XCA.

```
$ sudo apt install openssl
```

Set the adequate **basic constraints** for each certificate, the CA should have the **CA property**, the servers should have the **TLS Web Server Authentication property**, and (should you want to explore mTLS) the clients should have the **TLS Web Client Authentication property**. It is **mandatory** to put the hostname (i.e., fully qualified domain name – FQDN) in the common name (CN) of the server certificates. In our case, because we do not have a FQDN, we will be using the 'localhost' host name.

The CA's private key should not be distributed under any circumstance. The server private key (server_keys.pem) should only be used in the server. All certificates are public.

Edit a 'ca_extensions.cnf' file, and add to it the following content:

```
[Root_CA]
basicConstraints = critical, CA:TRUE
```

```
keyUsage = critical, keyCertSign

[TLS_Server]
basicConstraints = critical, CA:FALSE
keyUsage = critical, digitalSignature, keyEncipherment, dataEncipherment, keyAgreement
extendedKeyUsage = critical, serverAuth

[TLS_Client]
basicConstraints = critical, CA:FALSE
keyUsage = critical, digitalSignature, keyEncipherment, dataEncipherment, keyAgreement
extendedKeyUsage = critical, clientAuth
```

First, generate the CA's key pair (we will use EC cryptography, namely P-256) and we will not protect the private key with any password:

```
$ openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-384 -out ca_keys.pem
```

The file 'ca_keys.pem' will contain the CA's key pair. Now, we will create a CSR (Certificate Signing Request) from the CA's public key.

```
$ openssl req -new -key ca_keys.pem -out ca_csr.pem -subj "/CN=Root CA"
```

Once having our CSR, we will issue a self-certified certificate for the CA:

```
$ openssl x509 -signkey ca_keys.pem -in ca_csr.pem -req -days 365 -out ca.crt.pem -extensions Root_CA -extfile ca_extensions.cnf
```

Now, repeat the same sequence of openssl commands to generate a key pair, build a CSR and issue a certificate for the OpenVPN server:

```
$ openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-384 -out server_keys.pem
$ openssl req -new -key server_keys.pem -out server_csr.pem -subj "/CN=localhost"
```

but for issuing an end-user certificate we need to use a slightly different command. We need to use the CA certificate to get its name, the private key is not enough:

```
$ openssl x509 -CA ca.crt.pem -CAkey ca_keys.pem -CAcreateserial -in server_csr.pem -req -days 365 -out server.crt.pem -extensions TLS_Server -extfile ca_extensions.cnf
```

Now you have the crucial 'ca.crt.pem', 'server.crt.pem' and 'server_keys.pem' for TLS. You can pre-share the 'ca.crt.pem' with everyone, because all clients and servers need to trust this certificate beforehand. The 'server.crt.pem' is public and is usually sent by the server during the TLS handshake. The 'server_keys.pem' is secret and should only be known to the owner of the server.

3 Explore TLS

In order to use TLS in Python you must first create a TCP client-server and then enhance that socket with a TLS wrapper. 'ssl' is the Python module that allows wrapping the TCP socket with the TLS layer. The module's name comes from the protocol previous to TLS which was called Secure Sockets Layer (SSL). SSL is now insecure and should not be used in the wild. Nevertheless, for legacy reasons, often times you will find that people still use the name SSL to refer to the same concepts actually implemented by TLS.

Analyze the following server code (available as 'tls_server.py'):

```
import socket
import ssl

HOST = 'localhost'
PORT = 443
```

```

CERT = 'server.crt.pem'
KEY = 'server_keys.pem'

# context is fundamental for validation
# Note: the certification chain for the server is formed by only one certificate (below the root)

context = ssl.SSLContext( ssl.PROTOCOL_TLS_SERVER )
context.load_cert_chain( certfile=CERT, keyfile=KEY )

with socket.socket( socket.AF_INET, socket.SOCK_STREAM, 0 ) as sock:
    sock.bind( ( HOST, PORT ) )
    sock.listen( 5 )
    with context.wrap_socket( sock, server_side=True ) as tls_sock:
        print( "TLS server listening..." )
        conn, addr = tls_sock.accept()
        print( "Connection from: ", addr )
        data = conn.recv( 1024 )
        print( "Received:", data.decode() )
        conn.send( b"pong" )

```

NOTE: sometimes the server socket may not unbind properly from the port. You will need to wait for the TCP timeout before the server can be bound once again (i.e., before the server code can be run).

Analyze the corresponding client code (available as 'tls_client.py'):

```

import socket
import ssl

HOST = 'localhost'
PORT = 443

# context is fundamental for validation

context = ssl.create_default_context()
context.check_hostname = False # In this example we don't validate the certificate hostname against the hostname we are
                                using
context.verify_mode = ssl.CERT_NONE # In this example we don't validate the server certificate

with socket.create_connection( (HOST, PORT) ) as sock:
    with context.wrap_socket( sock, server_hostname=HOST ) as tls_sock:
        tls_sock.send( b"ping" )
        data = tls_sock.recv( 1024 )
        print( "Received:", data.decode() )

```

Check how easy it is to achieve a TLS tunnel between client-server on top of a regular TCP socket.

3.1 Alternative implementation with WolfSSL

Another possibility to explore TLS with Python is with the WolfSSL module.

```
$ sudo apt install libwolfssl-dev libtool autoconf
```

```

python3 -m venv venv
source venv/bin/activate
pip install wolfssl

```

With WolfSSL the server code may be the following:

```

import socket
import wolfssl

HOST = '127.0.0.1'
PORT = 443
CERT = 'server.crt.pem'
KEY = 'server_keys.pem'

context = wolfssl.SSLContext( wolfssl.PROTOCOL_TLS, server_side=True )
context.load_cert_chain( certfile=CERT, keyfile=KEY )

sock = socket.socket( socket.AF_INET, socket.SOCK_STREAM, 0 )

```

```

sock.bind(( HOST, PORT) )
sock.listen( 5 )

print( "TLS server listening..." )

conn, addr = sock.accept()
print( "Connection from:", addr )

tls_sock = context.wrap_socket( conn, server_side=True )

try:
    data = tls_sock.recv( 1024 )
    print( "Received:", data.decode() )
    tls_sock.send( b"pong" )
finally:
    tls_sock.close()
    conn.close()
    sock.close()

```

NOTE: sometimes the server socket may not unbind properly from the port. You will need to wait for the TCP timeout before the server can be bound once again (i.e., before the server code can be run).

The client code with WolfSSL may be the following:

```

import socket
import wolfssl

HOST = '127.0.0.1'
PORT = 443

context = wolfssl.SSLContext( wolfssl.PROTOCOL_TLS )
context.verify_mode = wolfssl.CERT_NONE # insecure but works

sock = socket.create_connection( (HOST, PORT) )
tls_sock = context.wrap_socket( sock, server_hostname=HOST )

try:
    tls_sock.send( b"ping" )
    data = tls_sock.recv( 1024 )
    print( "Received:", data.decode() )
finally:
    tls_sock.close()
    sock.close()

```

Check how easy it is to achieve a TLS tunnel between client-server on top of a regular TCP socket.

3.2 Observe TLS traffic

It is interesting to see the TLS messages that flow between client and server. Open another terminal and execute the following command to observe the traffic that flows through the loopback interface (lo):

```
$ tshark -i lo
```

Run a client-server interaction and see the TCP control segments and the TLS messages that flow between them. Can you identify the version of the TLS protocol that is being used by the applications? Can you identify the certificate that the server sends to the client?

Leave this packet capture tool running for debug.

4 Validate the certificate chain and constraints

Validate the certificate chain by reconfiguring the client context (this alternative is for the code that uses the SSL Python module):

```

context = ssl.SSLContext( ssl.PROTOCOL_TLS_CLIENT )
context.load_verify_locations( "ca.crt.pem" )

```

For the alternative using WolfTLS, do this:

```
# Require server certificate verification
context.verify_mode = wolfssl.CERT_REQUIRED

# Enable hostname check (SNI name must match CN/SAN)
context.check_hostname = True

# Load CA certificate(s) that signed your server certificate
context.load_verify_locations( cafile="ca.crt.pem" )
```

Did this simple chain validate properly? Note that the common name (CN) must match the name/address of the server you are connecting to.

Should it succeed at first try, select any CA certificate from `/etc/ssl/certs/` and check if the validation now fails.

You can further enhance TLS security by only allowing the latest version. The immediate implication is that only the latest and more secure ciphersuites will be allowed to establish the connection. Older versions exist for compatibility reasons and, unless a vulnerability is found, these are not insecure per se – they just do not offer the best protection and performance available in the state of the art. You can select the allowed version like this:

```
client_context.minimum_version = ssl.TLSVersion.TLSv1_3
client_context.maximum_version = ssl.TLSVersion.TLSv1_3
```

or, if using WolfTLS:

```
wolfssl.SSLContext(wolfssl.PROTOCOL_TLSv1_2)
```

You can also enforce further validations of basic constraints beyond the default validations.

5 Move to DTLS

You are advised to create a python virtual environment and install [PyDTLS](#) via `pip install python3-dtls`.

That is, for instance:

```
python3 -m venv venv
source venv/bin/activate
pip install python3-dtls
```

DTLS is very similar and uses the same context logic as in the `'ssl'` module, you just need to “patch” it to support UDP sockets:

```
from dtls import do_patch
do_patch()
```

However, there are two issues with this approach.

First, this Python module still uses an old version of libraries (`'libssl.so.1.1'` and `'libcrypto.so.1.1'`). This libraries were generated and are available in the resources provided in the resources for this class. You just have to copy them to the base directory where programs look for dynamic libraries (`'/usr/lib/x86_64-linux-gnu'`).

The second issue is that this patch apparently does not work with the current Python ssl module. The alternative is to use the native interface of the PyDTLS module.

Task 1: Create an UDP client-server. You have examples for this in the resources for this class.

Task 2: Verify that the previous certificate and certification chain validations are still working.

5.1 Alternative implementation with WolfSSL

Another possibility to explore DTLS with Python is with the WolfSSL module. Handling DTLS with WolfSSL is very similar to TLS and uses the same context logic, you just need to update the context.

Server:

```
import socket
import wolfssl

HOST = "127.0.0.1"
PORT = 443
CERT = "server.crt.pem"
KEY = "server.keys.pem"

context = wolfssl.SSLContext( wolfssl.PROTOCOL_DTLSv1_2, server_side=True )
context.load_cert_chain( certfile=CERT, keyfile=KEY )

sock = socket.socket( socket.AF_INET, socket.SOCK_DGRAM )
sock.bind(( HOST, PORT ) )

print( "DTLS server listening on", (HOST, PORT) )

# Grab first packet only to learn client address (do NOT decode)
_, client_addr = sock.recvfrom( 2048 )
print( "Connection from:", client_addr )

# Connect UDP socket so DTLS knows the peer
sock.connect( client_addr )

tls_sock = context.wrap_socket( sock, server_side=True )

try:
    # DTLS handshake
    tls_sock.do_handshake()
    print( "DTLS handshake complete." )

    # NOW receive real application data
    app_data = tls_sock.recv( 2048 )
    print( "Received:", app_data.decode() )

    tls_sock.send( b"pong" )

finally:
    tls_sock.close()
    sock.close()
```

Client:

```
import socket
import wolfssl

HOST = "127.0.0.1"
PORT = 443

context = wolfssl.SSLContext( wolfssl.PROTOCOL_DTLSv1_2 )
context.verify_mode = wolfssl.CERT_NONE # test only

sock = socket.socket( socket.AF_INET, socket.SOCK_DGRAM )
sock.connect( (HOST, PORT) )

# DTLS wrap
tls_sock = context.wrap_socket( sock, server_hostname=HOST )

try:
    # Handshake
    tls_sock.do_handshake()

    # Send ping
    tls_sock.send( b"ping" )

    # Receive pong
    data = tls_sock.recv( 2048 )
    print( "Received:", data.decode() )

finally:
    tls_sock.close()
    sock.close()
```

6 Benchmark

Measure the connection establishment time overhead, throughput, and latency without any encryption/authentication, with TLS (i.e., over TCP), and with DTLS (i.e., over UDP).

For the connection establishment time overhead you can make a coarse estimation by simply measuring the execution time of the client with the same message.

For the throughput measurement read a file of known size and measure the time it takes to transmit that file. Calculate the throughput in Mbit/s (or Gbit/s) by averaging the bits transmitted over the time in seconds.

For the latency measurement you can use the ‘time’ module and measure Round Trip Time (RTT) instead of one-way latency. Insert the current timestamp in the message sent to the server, have the server echo that message back, and simply subtract to current receive time the timestamp in the message.

Can you see any difference when using TLS/DTLS? Be sure to check connection establishment time both with small and larger messages. Can you derive any conclusion?

7 Acknowledgements

Authored by [André Zúquete](#), and [Vitor Cunha](#)

8 Bibliography

- [The TLS Protocol, Version 1.0](#)
- [Datagram Transport Layer Security Version 1.2](#)
- [ssl — TLS/SSL wrapper for socket objects](#)
- [Datagram Transport Layer Security for Python](#)